

# Lecture 2 — Knowledge (After Search)

---

## 1. Motivation: Reasoning with Knowledge

Humans reason by using what they already know and deriving new conclusions. Artificial Intelligence can do the same by **representing knowledge explicitly** and **reasoning logically** over it.

In this lecture, we study **how AI systems represent knowledge** and **how they infer new information** from existing facts.

---

## 2. Knowledge-Based Agents

A **knowledge-based agent** is an agent that reasons by operating on an **internal representation of knowledge**.

Instead of exploring paths like search agents, knowledge-based agents:

- Store facts about the world
- Use logic to draw conclusions
- Infer information that was not explicitly stated

Example: Ahmad Mohammad

Consider the following sentences:

1. If it didn't rain, Ahmad visited Majd today.
2. Ahmad visited Majd or Majed today, but not both.
3. Ahmad visited Majed today.

From these sentences we can infer:

- Ahmad did **not** visit Majd.
- If it had not rained, Ahmad would have visited Majd.
- Therefore, **it rained today**.

This conclusion is reached purely by logical reasoning.

### AI-Style Example

**Scenario:** An AI assistant monitors a smart home.

Facts:

- If the alarm is on, the house is empty.
- The house is not empty.

Inference:

- The alarm is not on.

## Code Example (Logic-based AI)

```
from logic import *

alarm = Symbol("alarm")
empty = Symbol("empty")

knowledge = And(
    Implication(alarm, empty),
    Not(empty)
)

print(model_check(knowledge, Not(alarm)))
```

Question?

**Q:** Can this AI reach the conclusion without searching states?

**Answer:** Yes. The AI reasons using logical inference over stored knowledge, not search.

---

## 3. Sentences and Propositional Logic

In this section, we explain **what kind of information AI can store as knowledge** and how we simplify real-world statements so that a computer can reason about them.

### Sentence

A **sentence** is an assertion about the world expressed in a knowledge representation language. A sentence must be either **true or false** — there is no middle value.

This is very important: **only sentences can be stored in a Knowledge Base.**

### Valid Sentences (True / False)

- "It is raining."
- "The server is down."
- "Ahmad visited Majed."
- "User 192.168.1.5 is authenticated."

Each of these can be evaluated as **true or false** at any moment in time.

### Not Sentences (Invalid for Logic)

- "Open the door!" (command)
- "Is it raining?" (question)
- "Please restart the system." (request)

These **cannot** be true or false, so AI logic systems cannot reason about them directly.

---

## AI Example: Cybersecurity Monitoring

Consider an AI system monitoring a network.

Possible sentences:

- "The firewall is active."
- "Port 22 is open."
- "An intrusion was detected."

Each of these is a sentence because it can be **true or false**, and therefore can be used for reasoning.

---

## Propositional Logic

**Propositional logic** is a way to represent sentences in a simplified, mathematical form so that computers can reason about them.

Each sentence is represented as a **proposition**.

A proposition must be **clearly true or false**. Ambiguous or uncertain statements are not valid propositions in propositional logic.

Example mapping:

| Real-world statement | Propositional symbol |
|----------------------|----------------------|
| It is raining        | P                    |
| It is Tuesday        | Q                    |
| Ahmad goes for a run | R                    |
| Firewall is active   | F                    |
| Intrusion detected   | I                    |

Once translated into symbols, AI no longer cares about the meaning — it only reasons about **truth values**.

---

## Code Example: Representing Sentences in AI

```
from logic import *

# Define propositional symbols
firewall = Symbol("firewall") # Firewall is active
intrusion = Symbol("intrusion") # Intrusion detected

# Knowledge base: if intrusion detected, firewall must be active
knowledge = Implication(intrusion, firewall)

# Query: is firewall active?
print(model_check(knowledge, firewall))
```

---

## Question?

**Q1:** Which of the following can be represented as a logical sentence?

- A. "Restart the server now"
- B. "The server is offline"
- C. "Did the attack succeed?"

**Correct Answer:**

- ☒ B only

---

## Key Takeaway

- AI can only reason about **statements that are true or false**
- These statements are called **sentences**
- Propositional logic is the first step to turning real-world knowledge into something an AI can reason about

---

## 4. Propositional Logic

Propositional logic represents knowledge using **propositions**, which are statements that are either true or false.

### Propositional Symbols

Propositions are represented by symbols, usually capital letters:

| Symbol | Meaning              |
|--------|----------------------|
| P      | It is raining        |
| Q      | It is Tuesday        |
| R      | Ahmad goes for a run |

The symbol itself has no meaning unless defined.

---

## 5. Logical Connectives

### NOT ( $\neg$ )

Negation reverses the truth value.

| P | $\neg P$ |
|---|----------|
|---|----------|

| P     | ¬P    |
|-------|-------|
| true  | false |
| false | true  |

Example: P = "It is raining", ¬P = "It is not raining".

AI Programming Example (NOT)

Scenario: An AI system checks whether a server is secure.

- P: "The server is vulnerable"
- ¬P: "The server is NOT vulnerable"

```
from logic import *  
  
vulnerable = Symbol("vulnerable")  
  
knowledge= Not(vulnerable)  
  
# Query: is the server not vulnerable?  
print(model_check(knowledge, Not(vulnerable)))
```

**Explanation :** If the AI knows the server is *not vulnerable*, it can safely conclude that any rule requiring a secure server is satisfied.

AND (∧)

AND is true only if both propositions are true.

| P     | Q     | P ∧ Q |
|-------|-------|-------|
| false | false | false |
| false | true  | false |
| true  | false | false |
| true  | true  | true  |

OR (∨) — Inclusive OR

OR is true if **at least one** proposition is true.

| P     | Q     | P ∨ Q |
|-------|-------|-------|
| false | false | false |
| false | true  | true  |

| P    | Q     | $P \vee Q$ |
|------|-------|------------|
| true | false | true       |
| true | true  | true       |

This is an **inclusive OR**, not XOR.

---

## When Do We Use OR in AI?

We use **OR** when **any one condition is sufficient** to trigger an action or make a conclusion.

Typical AI use cases:

- Multiple valid ways to authenticate a user
  - Multiple possible causes for an event
  - Multiple acceptable conditions to proceed
- 

## Simple Real-World Example

"A user can log in using a password **OR** a fingerprint."

- If password is correct → allow login
  - If fingerprint is valid → allow login
  - If both are valid → still allow login
- 

## AI Example: Authentication System

### Propositions:

- P: "Password is correct"
- Q: "Fingerprint is valid"

Logical rule:

$P \vee Q$

Meaning:

The user is authenticated if **either** method succeeds.

---

## Programming Example (OR)

```
from logic import *
```

```
password_ok = Symbol("password_ok")
fingerprint_ok = Symbol("fingerprint_ok")

# Knowledge base: login allowed if password OR fingerprint is valid
knowledge = Or(password_ok, fingerprint_ok)

# Query: can the user log in?
print(model_check(knowledge, password_ok))
```

Explanation

- OR does **not** mean "only one"
- OR means "at least one"
- This is why OR is useful in flexible AI decision systems

Question

**Q:** In which scenario should we use OR instead of AND?

- A. User must enter password and OTP
- B. User can log in using password or fingerprint

**Correct Answer:**

- ☒ B

Key Takeaway

Use **OR** (∨) when:

- Multiple conditions can independently satisfy a requirement
- The system should accept **any valid option**

Implication (→)

Implication represents “**if P, then Q**”. It is one of the most important connectives in AI because it models **rules, causality, and policies**.

| P     | Q     | P → Q |
|-------|-------|-------|
| false | false | true  |
| false | true  | true  |
| true  | false | false |
| true  | true  | true  |

If the antecedent (P) is false, the implication is **trivially true**.

---

## When Do We Use Implication in AI?

We use **Implication** when we want to express:

- Rules (IF condition THEN action)
- Cause–effect relationships
- Policies and constraints
- Security rules and alerts

Typical AI use cases:

- If an intrusion is detected → raise an alert
  - If the temperature is high → turn on cooling
  - If the user is authenticated → grant access
- 

## Simple Real-World Example

"If it is raining, then I carry an umbrella."

- Raining and umbrella → rule satisfied
  - Not raining → rule does not apply (still true)
  - Raining and no umbrella → rule violated
- 

## AI Example: Intrusion Detection System

### Propositions:

- P: "Intrusion detected"
- Q: "Security alert sent"

Logical rule:

$P \rightarrow Q$

Meaning:

Whenever an intrusion is detected, the AI must send an alert.

---

## Programming Example (Implication)

```
from logic import *  
  
intrusion = Symbol("intrusion")
```



```

alert = Symbol("alert")

# Knowledge base: if intrusion is detected, an alert must be sent
knowledge = Implication(intrusion, alert)

# Query: does the system require an alert?
print(model_check(knowledge, alert))

```

---

## Explanation

- Implication models **rules**, not events
  - The rule only breaks when:
    - The condition is true
    - But the result is false
  - If the condition never happens, the rule is still logically valid
- 

## Question

**Q:** Which of the following best fits an implication rule?

- A. User must enter password **and** OTP
- B. User can log in with password **or** fingerprint
- C. If malware is detected, the system blocks the file

**Correct Answer:**

- ☒ C
- 

## Key Takeaway

Use **Implication** ( $\rightarrow$ ) when:

- You want to encode rules and policies
  - One condition **enforces** another outcome
  - You are modeling cause  $\rightarrow$  effect in AI systems
- 

## Biconditional ( $\leftrightarrow$ )

A biconditional means “**if and only if**”. It expresses a **two-way rule**: both sides depend on each other.

$$P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$$

| P | Q | P $\leftrightarrow$ Q |
|---|---|-----------------------|
|---|---|-----------------------|

---

| P     | Q     | $P \leftrightarrow Q$ |
|-------|-------|-----------------------|
| false | false | true                  |
| false | true  | false                 |
| true  | false | false                 |
| true  | true  | true                  |

When Do We Use Biconditional in AI?

We use **biconditional** when:

- Two conditions must **always occur together**
- One condition is true **exactly when** the other is true
- We want to express **equivalence**, not just a rule

Typical AI use cases:

- Access is granted **if and only if** the user is authenticated
- A system is safe **iff** all security checks pass
- A student passes **iff** the final grade  $\geq 50$

Simple Real-World Example

"A user can access the system **if and only if** they are authenticated."

This means:

- If the user is authenticated  $\rightarrow$  access is granted
- If access is granted  $\rightarrow$  the user must be authenticated

AI Example: Secure Access Control

Propositions:

- P: "User is authenticated"
- Q: "Access is granted"

Logical rule:

$P \leftrightarrow Q$

Meaning:

Authentication and access must always match.

## Programming Example (Biconditional)

```
from logic import *

authenticated = Symbol("authenticated")
access = Symbol("access")

# Access is granted if and only if the user is authenticated
knowledge = Biconditional(authenticated, access)

# Query: is access granted exactly when authentication is true?
print(model_check(knowledge, access))
```

---

### Explanation

- Biconditional = **two implications together**
- Stronger than implication
- Very strict: if one side is false, the other must also be false

---

### Question

**Q:** Which statement requires a biconditional rule?

- A. If malware is detected, the system blocks the file
- B. User can log in using password or fingerprint
- C. A user has access **if and only if** they are authenticated

**Correct Answer:**

- ☒ C

---

### Key Takeaway

Use **Biconditional** ( $\leftrightarrow$ ) when:

- Two conditions must always be true together
- You want **equivalence**, not just cause  $\rightarrow$  effect

---

## 6. Models

A **model** is an assignment of truth values to every propositional symbol. Each model represents **one possible world** that the AI considers.

Example:

```
P = true
Q = false
```

This model means that proposition P is true in the world, while Q is false.

With  $n$  propositions, there are  **$2^n$  possible models**.

---

## When Do We Use Models in AI?

Models are used when an AI system needs to:

- Consider **different possible situations** of the world
- Check whether a conclusion is true in **all possible cases**
- Reason under uncertainty before making a decision

Typical AI use cases:

- Cybersecurity: checking all possible attack scenarios
- Robotics: considering all possible sensor states
- Game AI: evaluating all possible board configurations

---

## Simple Real-World Example

Suppose an AI monitors weather and traffic:

- P: "It is raining"
- Q: "Traffic is heavy"

Possible models:

| Model | P (Raining) | Q (Traffic) |
|-------|-------------|-------------|
| M1    | true        | true        |
| M2    | true        | false       |
| M3    | false       | true        |
| M4    | false       | false       |

Each row is a **different possible world** the AI may reason about.

---

## AI Example: Security Monitoring

**Scenario:** An AI monitors a server.

Propositions:

- A: "An attack is occurring"

- F: "Firewall is enabled"

The AI considers different models to decide what actions are safe.

Example model:

```
A = true
F = true
```

Meaning: An attack is happening, but the firewall is active.

---

## Programming Example (Models)

```
from logic import *

attack = Symbol("attack")
firewall = Symbol("firewall")

# Example model: attack is true, firewall is false
model = {
    attack: True,
    firewall: False
}

print("Attack:", model[attack])
print("Firewall:", model[firewall])
```

---

## Explanation

- A **model** is not a rule, it is a **snapshot of the world**
  - AI does not know which model is real, so it reasons about **all of them**
  - Inference checks what is true in **every valid model**
- 

## Question

**Q:** If an AI has 3 propositional symbols, how many models are possible?

A. 3 B. 6 C. 8 D. 9

**Correct Answer:**

- ☒ C ( $2^3 = 8$ )
- 

## Key Takeaway

Use **models** to:

- Represent possible worlds
  - Test whether conclusions always hold
  - Support logical inference in AI systems
- 

## 7. Knowledge Base (KB)

A **knowledge base (KB)** is a set of sentences that are assumed to be **true**. It represents **everything the AI knows** about the world at a given moment.

Think of the knowledge base as the AI's **memory + rules**.

Example:

```
P
¬Q
(P ∧ ¬Q) → R
```

This means:

- P is true
  - Q is false
  - If P is true and Q is false, then R must be true
- 

### When Do We Use a Knowledge Base in AI?

We use a **knowledge base** when:

- We want the AI to store **facts** and **rules** together
- We want to **reason**, not just react
- We want conclusions to be derived automatically

Typical AI use cases:

- Expert systems (medical diagnosis, troubleshooting)
  - Cybersecurity monitoring systems
  - Smart assistants and decision-support systems
- 

### Simple Real-World Example

Suppose an AI helps decide whether a building is safe.

Facts:

- P: "Fire alarm is active"
- Q: "Sprinklers are active"

- R: "Building must be evacuated"

Rule:

$$(P \wedge Q) \rightarrow R$$

Meaning:

If the fire alarm and sprinklers are active, the building must be evacuated.

---

## AI Example: Cybersecurity Policy Engine

**Scenario:** An AI enforces security policies.

Propositions:

- A: "User is authenticated"
- P: "User has permission"
- G: "Access is granted"

Rules stored in the KB:

```
A
P
(A ∧ P) → G
```

Meaning:

- The user is authenticated
  - The user has permission
  - If both are true, access must be granted
- 

## Programming Example (Knowledge Base)

```
from logic import *

# Define symbols
authenticated = Symbol("authenticated")
permission = Symbol("permission")
access = Symbol("access")

# Build the knowledge base
knowledge = And(
    authenticated,
    permission,
    Implication(And(authenticated, permission), access)
```

```
)  
  
# Query: should access be granted?  
print(model_check(knowledge, access))
```

---

## Explanation

- A **knowledge base** contains both **facts** and **rules**
- Facts describe the current situation
- Rules describe how new facts can be inferred
- The AI does not manually decide — logic does

---

## Question

**Q:** Which of the following belongs in a Knowledge Base?

- A. A search algorithm (BFS)
- B. A fact like "The server is offline"
- C. A user interface button

**Correct Answer:**

- ☒ B

---

## Key Takeaway

Use a **Knowledge Base (KB)** when:

- You want AI to **store knowledge explicitly**
- You want conclusions to be **derived automatically**
- You want reasoning instead of hard-coded decisions

---

## 8. Entailment ( $\models$ )

**Entailment** describes when some knowledge **logically guarantees** another fact.

Formally:

$$\alpha \models \beta$$

means:

In **every model** where  $\alpha$  is true,  $\beta$  is also true.

Entailment is a **semantic relationship**, not a logical connective.

---



## When Do We Use Entailment in AI?

We use **entailment** when we want to check:

- Whether a conclusion is **always true** given what the AI knows
- Whether the AI is **allowed to infer** a fact
- Whether a rule **logically follows** from the knowledge base

Typical AI use cases:

- Expert systems ("Does the diagnosis necessarily follow?")
- Security analysis ("Does this configuration imply a vulnerability?")
- Formal verification and policy checking

---

## Simple Real-World Example

Statement:

"It is a Tuesday in January"

What does this entail?

- "It is January" ☒ (entailed)
- "It is raining" ☐ (not entailed)

So:

```
TuesdayInJanuary  $\models$  January  
TuesdayInJanuary  $\not\models$  Raining
```

---

## AI Example: Access Control Reasoning

**Scenario:** An AI checks whether a user must be granted access.

Propositions:

- A: "User is authenticated"
- P: "User has permission"
- G: "Access is granted"

Knowledge Base:

```
A  
P  
(A  $\wedge$  P)  $\rightarrow$  G
```

Question:

Does  $KB \models G$  ?

Answer:

- Yes. In **every possible model** where A and P are true, G must also be true.
- 

## Programming Example (Entailment via Model Checking)

```
from logic import *

# Define symbols
authenticated = Symbol("authenticated")
permission = Symbol("permission")
access = Symbol("access")

# Knowledge base
knowledge = And(
    authenticated,
    permission,
    Implication(And(authenticated, permission), access)
)

# Entailment check: does KB entail access?
print(model_check(knowledge, access))
```

If this prints **True**, it means:

In **all models** where the KB is true, **access** is also true.

---

## Interface-Level View (How AI Uses Entailment)

```
INPUT:
- Knowledge Base (facts + rules)
- Query (what we want to know)

PROCESS:
- Model Checking / Inference

OUTPUT:
- Yes (entailed) or No (not entailed)
```

This is exactly how expert systems, policy engines, and reasoning modules work internally.

---

## How Model Checking Implements Entailment

1. Generate **all possible models**

2. Discard models where the KB is false
3. Check the query in the remaining models

If the query is true in **all** of them → entailed.

This connects entailment directly to **model checking**.

---

## Explanation

- Entailment asks: "**Is this conclusion unavoidable?**"
- It depends on **all possible worlds**, not just one
- If the conclusion could ever be false → no entailment

Think of it as:

"Given everything I know, could this ever be false?"

If the answer is **no**, then it is entailed.

---

## Question

**Q:** Which statement is correct?

- A.  $P \rightarrow Q$  means  $P \models Q$
- B. If  $KB \models Q$ , then  $Q$  is true in all valid models
- C. Entailment depends on a single model

**Correct Answer:**

- ☒ B
- 

## Key Takeaway

Use **Entailment ( $\models$ )** when:

- You want guaranteed conclusions
  - You want to verify AI reasoning
  - You want logic-based correctness, not guessing
- 

## 9. Inference

**Inference** is the process of deriving **new sentences** from existing ones using logic.

In other words:

Inference is how an AI system *thinks* using what it already knows.

Inference does **not** add random facts. It only produces conclusions that are **logically guaranteed** by the Knowledge Base.

---

## When Do We Use Inference in AI?

Inference is used whenever an AI system must:

- Draw conclusions automatically
- Answer questions based on stored knowledge
- Make decisions without hard-coding all cases

Typical AI use cases:

- Medical diagnosis (symptoms → disease)
  - Cybersecurity (events → attack detection)
  - Game logic (moves → win/lose)
- 

## Simple Real-World Example

Facts:

- P: "It is Tuesday"
- Q: "It is raining"
- R: "Ahmad goes for a run"

Rule:

$$(P \wedge \neg Q) \rightarrow R$$

Meaning:

If it is Tuesday and it is **not** raining, Ahmad goes for a run.

Additional facts:

P  
 $\neg Q$

**Inference result:**

R

We did not store R directly — it was **inferred**.

---

## AI Example: Rule-Based Decision System

**Scenario:** An AI fitness assistant.

Propositions:

- T: "Today is Tuesday"
- R: "It is raining"
- G: "Go running"

Rule:

$$(T \wedge \neg R) \rightarrow G$$

If the AI knows today is Tuesday and it is not raining, it infers that the user should go running.

---

## Programming Example (Inference)

```
from logic import *

# Define symbols
tuesday = Symbol("tuesday")
raining = Symbol("raining")
go_run = Symbol("go_run")

# Knowledge base
knowledge = And(
    tuesday,
    Not(raining),
    Implication(And(tuesday, Not(raining)), go_run)
)

# Query: should the user go running?
print(model_check(knowledge, go_run))
```

## Explanation

- Inference creates **new knowledge** from old knowledge
  - The AI never guesses
  - Every inferred sentence must logically follow from the KB
- 

## Key Takeaway

Inference is the **engine of reasoning** in logic-based AI systems.

---

## 10. Model Checking

To determine whether  $KB \models \alpha$ :

1. Enumerate all possible models
2. Keep models where KB is true
3. Check whether  $\alpha$  is true in all those models

### Python Example

```
from logic import *

rain = Symbol("rain")
Majd = Symbol("Majd")
Majed = Symbol("Majed")

knowledge = And(
    Implication(Not(rain), Majd),
    Or(Majd, Majed),
    Not(And(Majd, Majed)),
    Majed
)

model_check(knowledge, rain)
```

---

## 11. Knowledge Engineering

Scenario: Smart Building Evacuation System

An AI system decides whether to evacuate a building based on sensor data.

Propositions:

- F: Fire detected
- S: Smoke detected
- A: Alarm is triggered
- E: Building is evacuated

Rules

- If fire OR smoke is detected  $\rightarrow$  alarm is triggered
- If alarm is triggered  $\rightarrow$  evacuate the building

Logical representation:

$(F \vee S) \rightarrow A$

A → E

```
from logic import *

fire = Symbol("fire")
smoke = Symbol("smoke")
alarm = Symbol("alarm")
evacuate = Symbol("evacuate")

knowledge = And(
    Implication(Or(fire, smoke), alarm),
    Implication(alarm, evacuate),
    fire # assume fire is detected
)

# Query: should the building be evacuated?
print(model_check(knowledge, evacuate))
```

Explanation:

- The AI encodes rules about emergencies
- Sensor readings become facts
- Inference derives new actions automatically
- No if/else logic is hard-coded

This example shows how knowledge engineering means:

- Choosing the right propositions
- Writing correct logical rules
- Letting inference produce decisions

---

## 12. Inference Rules

**Inference rules** allow an AI system to derive new knowledge **without checking every possible model**.

Instead of enumerating all models (as in Model Checking), inference rules apply **local logical steps** that humans naturally use when reasoning.

Each inference rule has:

- **Premise** (what we already know)
- **Conclusion** (what we can safely infer)

They are usually written with a horizontal line separating premise and conclusion.

---

## Modus Ponens

$$\begin{array}{l} P \rightarrow Q \\ P \\ \hline Q \end{array}$$

Definition: If a rule is true and its condition is true, then the result must be true.

### When do we use it?

- When we know a **rule** and we know the **condition is true**.
- When we have an IF-THEN rule and we know the IF part happened

### Simple Example

- If it is raining  $\rightarrow$  Ahmad is inside
- It is raining
- Therefore: Ahmad is inside

### AI Example

Intrusion detection:

- If malware is detected  $\rightarrow$  block the file
- Malware is detected
- Therefore: block the file

### Programming Example

```
from logic import *

malware = Symbol("malware")
block = Symbol("block")

knowledge = And(
    Implication(malware, block),
    malware
)

print(model_check(knowledge, block))
```

**Note:** *rule + condition = result.*

---

## De Morgan's Laws



$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

Definition: Negation can be pushed inside expressions by switching AND and OR.

### When do we use them?

To **push negation inward**, which is required for CNF.

### Simple Example

- Not (Ahmad and Ron passed)
- Means: Ahmad did not pass OR Ron did not pass

## Distributive Property

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$$

$$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$$

Definition: Logical expressions can be expanded by distributing AND over OR (and vice versa).

### When do we use it?

To **break complex expressions into smaller clauses**, especially when converting to CNF.

### Teaching Insight

Explain this like algebra:

$$a(b + c) = ab + ac$$

Logic works the same way.

## 13. Knowledge and Search Problems

Inference can be viewed as a **search problem**, where the AI searches for a *proof* instead of a path.

### Mapping Inference to Search

| Search Concept   | Inference Meaning                                |
|------------------|--------------------------------------------------|
| Initial state    | Initial Knowledge Base                           |
| Actions          | Inference rules (e.g., Modus Ponens, Resolution) |
| Transition model | Knowledge Base after applying an inference rule  |

| Search Concept | Inference Meaning                    |
|----------------|--------------------------------------|
| Goal test      | Has the query been logically proven? |
| Path cost      | Number of inference steps            |

**Notes:**

In classical search we look for a *path*. In logical inference we look for a *proof*.

This connection shows how **search algorithms reappear in reasoning systems**, but operate over *knowledge* instead of *states*.

---

## 18. Summary

- Represent knowledge explicitly
- Reason using logical rules
- Infer new facts from existing knowledge
- Use search principles to perform logical reasoning